



CGNS Proposal for Extension 0051

Implicit FaceCenter Indexing for NGON-Based Unstructured Zones

Version 1.2
July 2, 2026

CPEX#	0051
Scope	Drop the <code>PointList/PointRange</code> requirement for <code>FaceCenter</code> data on complete <code>NGON_n</code> zones
Champion	M. Scot Breitenfeld
Organization	The HDF Group
E-mail	brtnfld@hdfgroup.org
GitHub Issue	TBD (implementation gaps tracked as #964 , #965)
Reference Impl.	TBD
Target Release	CGNS v5.0.0 (major release)
Date First Posted	July 2, 2026
SIDS Status	proposed
Filemap Status	proposed
MLL Status	proposed

Contents

1	Abstract	3	10
2	Motivation and Rationale	3	11
2.1	Problem Statement	3	12
2.2	Origin of the Restriction	4	13
2.3	Why <code>NGON_n</code> Changes the Picture	5	14
2.4	Precedent: <code>CellCenter</code> Already Works This Way	5	15
2.5	Existing Implementation Gaps	6	16

2.5.1	<code>cg_sol_write</code> rejects plain <code>FaceCenter</code>	6	17
2.5.2	<code>cgnscheck</code> never reads <code>FlowSolution_t</code> 's point set	7	18
3	Detailed Description	8	19
3.1	Design Overview	8	20
3.2	Definition: Complete NGON Face Coverage	8	21
3.3	Revised SIDS Text: <code>FlowSolution_t</code> (§7.7, Note 5)	9	22
3.4	Revised SIDS Text: <code>DiscreteData_t</code> (§12.4)	9	23
3.5	Revised <code>DataSize[]</code> Structure Function	10	24
3.6	MLL Changes	11	25
3.6.1	New internal helper	11	26
3.6.2	Fix: <code>cg_sol_write</code> allow-list	12	27
3.6.3	Fix: <code>cgnscheck</code> point-set-aware sizing	13	28
3.6.4	No change to on-disk representation	13	29
3.7	Scope / Non-Goals	13	30
4	Backward Compatibility	14	31
5	Examples	15	32
5.1	Example 1: Before — Identity <code>PointRange</code> (current, still valid) . . .	15	33
5.2	Example 2: After — Implicit <code>FaceCenter</code> (this CPEX)	16	34
5.3	Example 3: Reading — Dispatching on Coverage	17	35
5.4	Example 4: Rejected — Non-Contiguous NGON_n Sections	17	36
5.5	Example 5: Accepted — NGON_n Coexisting with a Boundary-Patch Section	18	38
6	SIDS Impact	18	39
7	File Mapping Impact	19	40
8	Test Plan	19	41
9	Design Decisions	20	42
9.1	Alternative 1: Broaden Coverage to Any Complete 2-D Section . . .	20	43
9.2	Alternative 2: New Explicit Marker Node (e.g., <code>FacesComplete</code>) . .	20	44
9.3	Alternative 3 (chosen): Structural Inference from NGON_n Coverage .	21	45
9.4	On Scope: Why Not Also Amend <code>ZoneSubRegion_t</code> ?	21	46
10	Open Questions	21	47
11	Summary of API Additions	21	48
12	References	22	49

1 Abstract

SIDS Section 7.7 (`FlowSolution_t`), Note 5, states that for unstructured zones `GridLocation` is restricted to `Vertex` or `CellCenter` unless a `PointList` or `PointRange` is also present. The same restriction appears, near word-for-word, in Section 12.4 (`DiscreteData_t`). This forces every `FaceCenter` solution on an unstructured zone to carry an explicit point set — even when the data spans literally every face in the zone, in which case the point set is the identity permutation and conveys no information beyond what is already implied by the zone’s own element numbering.

For zones whose faces are described by `NGON_n` elements, this identity point set is redundant: SIDS Section 7.3 already guarantees that `NGON_n` elements “specify all the faces in the grid,” giving faces the same kind of complete, contiguous, globally unique numbering that cells already have — which is precisely why `CellCenter` never needed a `PointList/PointRange` in the first place.

This CPEX proposes a narrow, precisely scoped relaxation: when an unstructured zone has *complete NGON face coverage* (defined in Section 3.2), `GridLocation = FaceCenter` data may be written and read *without* a `PointList/PointRange`, indexed directly against the zone’s `NGON_n` element numbering, exactly as `CellCenter` is indexed against the zone’s `CellDimension-order` elements. The relaxation is strictly additive: files using the existing identity-point-set workaround remain fully conformant, and the change is scoped to `NGON_n` coverage only — it explicitly does *not* extend to classical 2-D element sections (`TRI_3`, `QUAD_4`, ...), which are conventionally used for boundary patches only and do not carry SIDS’s “all faces” guarantee.

While researching the current behavior, two independent implementation gaps were found that this CPEX’s reference implementation also addresses: `cg_sol_write` cannot create a `FaceCenter`-located solution directly (Section 2.5.1), and `cgnscheck` never reads a `FlowSolution_t`’s `PointList/PointRange` when validating data-array sizes (Section 2.5.2), so today’s identity-point-set files are not actually verifiable by the reference conformance tool.

2 Motivation and Rationale

2.1 Problem Statement

A common CFD data-storage need is a quantity defined on *every* face of an unstructured mesh (e.g., face-normal velocities, surface fluxes on all element boundaries). For a zone built from `NFACE_n/NGON_n` polyhedral connectivity, the following tree is representative:

```

1 fluid_zone Zone_t [[27 8 0]]
2 +---ZoneType ZoneType_t "Unstructured"
3 +---NGonElements Elements_t [22 0]
4 | +---ElementRange IndexRange_t [ 1 36]
5 | :
6 +---NFaceElements Elements_t [23 0]
7 | +---ElementRange IndexRange_t [37 44]
8 | :
9 +---CellData FlowSolution_t
10 | +---GridLocation GridLocation_t "CellCenter"
11 | +---Pressure DataArray_t /* 8 values -- no PointList/PointRange */
12 +---FaceData FlowSolution_t
13 | +---GridLocation GridLocation_t "FaceCenter"
14 | +---NormalX DataArray_t /* 36 values */
15 | :
16 | +---PointRange IndexRange_t [[1, 36]]
17 | ~~~~ identity range -- required by Note 5, conveys no new
    information

```

CellData needs no point set: GridLocation = CellCenter alone is sufficient, because the zone’s cells are already uniquely and completely numbered by its volume Elements_t sections. FaceData, however, is required by SIDS Note 5 to carry a PointRange, even though [1,36] is exactly the NGonElements section’s own ElementRange — information the reader already has. Two consequences follow:

1. **Confusion.** Users and tool authors reasonably ask why an identity point set is mandatory for one GridLocation and forbidden (well, unnecessary) for another, when both are governed by the same element-numbering machinery.
2. **Detection cost.** Any code path that wants to special-case “this solution spans every face” — to avoid gather/scatter overhead in a parallel read, for instance — must first read the point set and compare it against the face count, on every zone, for every solution. This is a small but repeated and entirely avoidable per-solution check.

2.2 Origin of the Restriction

Note 5 was introduced by CPEX 0031 (“revamped usage of PointList and PointRange”), incorporated into SIDS v3.1. The SIDS gives its own rationale immediately following the notes list in Section 7.7:

“For unstructured grids, the value of GridLocation alone specifies location and indexing of flow solution data only for vertex and cell-centered data. The reason for this is that element-based grid connectivity provided in the Elements_t data structures explicitly indexes only vertices and cells. For data stored at alternate grid locations (e.g., edges), additional connectivity information is needed. This is provided by the optional fields PointRange and PointList...”

This is a correct and still-necessary rule for the *general* case: an unstructured zone is only ever *required* to number its vertices and its CellDimension-order ele-

ments. A zone built purely from TETRA_4/PYRA_5/PENTA_6/HEXA_8/ MIXED sections, with no face-type Elements_t section at all, has no face numbering whatsoever — FaceCenter is genuinely meaningless there without an explicit point set.

Notably, SIDS v3.1 is also the version in which NFACE_n was formalized and the NGON_n description was substantially revised (per the same changelog entry). The two changes were made in the same release but were not cross-referenced: Note 5 was written for, and still correctly describes, the general classical-element case, but it was never revisited to carve out the NGON_n case where a complete face numbering is guaranteed to exist.

2.3 Why NGON_n Changes the Picture

SIDS Section 7.3 (Elements_t) states two facts that together give faces the same numbering guarantee that cells already enjoy:

“The elements are indexed with a global numbering system, starting at 1, for all element sections under a given Zone_t data structure. The global numbering insures that each element, whether it’s a cell, a face, or an edge, is uniquely identified by its number. They are also listed as a continuous list of element numbers within any single element section.”

*“Arbitrary polyhedral elements may be defined using the NGON_n and NFACE_n element types. **The NGON_n element type is used to specify all the faces in the grid,** and the NFACE_n element type is then used to define the polyhedral elements as a collection of these faces.”*

Combined, these two statements mean that whenever a zone’s volume cells are described with NFACE_n/NGON_n connectivity, the union of that zone’s NGON_n ElementRange(s) is — by the SIDS’s own existing guarantees, with no new machinery required — a complete, contiguous, globally unique numbering of every face in the zone. This is structurally identical to how a zone’s volume Elements_t sections already give CellCenter an implicit, unambiguous numbering.

2.4 Precedent: CellCenter Already Works This Way

SIDS Section 7.7 defines the DataSize[] structure function, which is the authoritative statement of how FlowSolution_t array sizes are derived. Quoted verbatim from the current SIDS source (non-Rind branches; the Rind-present branch follows the same pattern):

```

1  if (PointRange/PointList is present) then
2  {
3      DataSize[] = ListLength[] ;
4  }
5  else if (Rind is absent) then
6  {
7      if (GridLocation = Vertex) or (GridLocation is absent)
8      {
9          DataSize[] = VertexSize[] ;
10     }
11     else if (GridLocation = CellCenter) then
12     {
13         DataSize[] = CellSize[] ;
14     }
15 }

```

The `CellCenter` branch already relies *entirely* on implicit element-based sizing: no `PointList/PointRange` is required there, because `CellSize[]` is already well-defined from the zone's `CellDimension`-order `Elements_t` sections. This CPEX asks for nothing more than extending that same, already-accepted pattern to `FaceCenter` in the one case where an equally strong SIDS guarantee already exists — `NGON_n` coverage. See Section 3.5 for the precise proposed addition to `DataSize[]`.

2.5 Existing Implementation Gaps

Two related gaps in the reference implementation were found while preparing this proposal. Neither is created by this CPEX, but both must be fixed as part of its reference implementation, since they sit directly in the code paths this CPEX touches.

2.5.1 `cg_sol_write` rejects plain `FaceCenter`

`cg_sol_write` (`cgnslib.c`) validates its `location` argument against an explicit allow-list:

```

1  if (location != CGNS_ENUMV(Vertex) &&
2      location != CGNS_ENUMV(CellCenter) &&
3      location != CGNS_ENUMV(IFaceCenter) &&
4      location != CGNS_ENUMV(JFaceCenter) &&
5      location != CGNS_ENUMV(KFaceCenter)) {
6      cgi_error("Given grid location not supported for FlowSolution_t");
7      return CG_ERROR;
8  }

```

Plain `FaceCenter` — the only location an unstructured zone can use for face data — is not in this list. This does *not* block the *explicit*-point-set form in Example 5.1: `cg_sol_ptset_write()` already creates a `FaceCenter` solution with an accompanying `PointList/PointRange` in a single call today, because it validates via `cgi_check_location` directly and, internally, works around `cg_sol_write`'s allow-list by calling

it with `location = Vertex` and overwriting `sol->location` afterward — itself 162
telling evidence that the allow-list is redundant with `cgi_check_location` rather 163
than an independent rule (see Section 3.6.2). The gap that *does* block today is 164
the *point-set-free* form this CPEX introduces: `cg_sol_ptset_write()` requires 165
`npnts > 0`, so it cannot be used to create a bare `FaceCenter` solution with no point 166
set at all, and `cg_sol_write` is the only entry point for that shape — which is 167
exactly where the allow-list rejects it. Today, the only way to create a point-set- 168
free `FaceCenter` solution is the two-step sequence `cg_sol_write(..., CellCenter,` 169
`...)` followed by `cg_gridlocation_write(..., FaceCenter)`, which happens to 170
work only because `cg_gridlocation_write` performs its own, separate validation 171
(`cgi_check_location`) that *does* permit `FaceCenter` for `CellDimension ≥ 3`. This 172
is a latent usability bug independent of Note 5, and it directly affects every user 173
who wants to write a point-set-free `FaceCenter` solution today (e.g. the identity- 174
PointRange workaround plus later `cg_gridlocation_write` override, or any future 175
use of this CPEX’s implicit form), whether or not this CPEX is adopted. Tracked 176
as [CGNS/CGNS #964](#). 177

2.5.2 cgnscheck never reads FlowSolution_t’s point set 178

`check_solution()` (`cgnscheck.c`) computes the expected data-array size via `get_-` 179
`data_size()`, which contains: 180

```
1 if (z->type == CGNS_ENUMV(Unstructured)) {
2     error ("grid location %s not valid for unstructured zone",
3         cg_GridLocationName (location));
4     return 0;
5 }
```

This branch is checked *before* `get_data_size()` ever looks at `GridLocation = FaceCenter`, 182
so it fires for *any* `GridLocation` other than `Vertex/CellCenter` on an unstructured 183
zone — including `FaceCenter` with a fully populated, conformant `PointRange`. 184
`check_solution()` never calls `cg_ptset_info/cg_ptset_read` at all, so today’s 185
required identity-point-set workaround (Section 2.1) is not merely *unvalidated* by 186
the reference conformance checker — it is *actively misreported*. `error()` is non-fatal 187
but not silent: it prints a counted `ERROR:` line and increments `cgnscheck`’s error 188
tally for every `FaceCenter` solution on every unstructured zone in the file, today, 189
for files that are fully conformant under the *current* SIDS. This is a stronger 190
motivation than mere non-validation: it is a false-positive in the reference tool that 191
this CPEX’s fix also happens to resolve. This CPEX’s reference implementation 192
fixes `get_data_size()` to honor `ListLength[]` from an explicit point set (already 193
required by the current SIDS) in addition to the new implicit `NGON_n` sizing this 194
CPEX adds. Tracked as [CGNS/CGNS #965](#).

3 Detailed Description

3.1 Design Overview

The proposal has four parts:

1. A formal SIDS definition of *complete NGON face coverage* (Section 3.2), built entirely from existing SIDS guarantees — no new node types or structural elements are introduced.
2. A redline amendment to SIDS Note 5 in `FlowSolution_t` (§7.7) and the matching clause in `DiscreteData_t` (§12.4), and a corresponding addition to the `DataSize[]` structure function (Section 3.5).
3. MLL changes (Section 3.6) to compute and validate the implicit face count, expose it to readers via a new public accessor (Section 3.6.1), and fix the two unrelated implementation gaps from Section 2.5 (as standalone, independently backportable patches — see Section 11).
4. A test plan (Section 8) covering the coverage predicate, the new sizing rule, and the write-ordering constraint this CPEX introduces (Section 4).

3.2 Definition: Complete NGON Face Coverage

An unstructured zone Z has **complete NGON face coverage** if and only if:

1. Z contains one or more `Elements_t` sections of type `NGON_n`; and
2. the union of the `ElementRanges` of those sections forms a single contiguous block $[E_{\text{start}}, E_{\text{end}}]$ of the zone’s global element numbering (trivially true for one section; for multiple `NGON_n` sections, their ranges must partition $[E_{\text{start}}, E_{\text{end}}]$ with no gaps, and no element index inside that block may belong to a non-`NGON_n` section).

When these conditions hold, the face count is $N_{\text{face}} = E_{\text{end}} - E_{\text{start}} + 1$, and the **implicit FaceCenter indexing rule** is: the j -th entry (1-based) of the `DataArray_t` maps to global element index

$$E_{\text{start}} + j - 1, \quad 1 \leq j \leq N_{\text{face}}.$$

This rule refers *only* to the $[E_{\text{start}}, E_{\text{end}}]$ block; it says nothing about, and is unaffected by, any other `Elements_t` sections elsewhere in Z ’s global numbering. In particular, a zone may freely contain separate boundary-patch sections (`TRI_3`, `QUAD_4`, ...) used for `BC_t` definitions *outside* the `NGON_n` block — a common pattern in files where the volume mesh is polyhedral but boundary patches are still exposed via classical elements for tool compatibility. Their presence does not disqualify Z from complete

NGON face coverage and does not change what the implicit indexing rule means; those sections simply are not candidates for it. This is a deliberate relaxation from an earlier draft of this CPEX, which excluded any zone containing such sections; that exclusion turned out to be unnecessary, since TRI_3/QUAD_4 sections were never valid candidates for “all faces” in the first place (see below) — the only thing that matters is that $[E_{\text{start}}, E_{\text{end}}]$ itself is unambiguous, which condition 2 already guarantees regardless of what else exists in the zone.

This CPEX intentionally does **not** extend the implicit-sizing idea to classical 2-D sections (TRI_3, QUAD_4, ...) themselves, i.e., there is no equivalent rule that lets *their* combined `ElementSize` stand in for “the face count of the zone.” Unlike `NGON_n`, the SIDS gives no guarantee that such sections enumerate *all* faces of a zone; in the overwhelming majority of existing CGNS files they enumerate boundary faces only. Treating their combined `ElementSize` as “the face count of the zone” would silently misinterpret a large fraction of legacy files. This CPEX similarly does not extend to `EdgeCenter` on 2-D zones: SIDS has no analogous “this element type enumerates all edges” primitive for edges, so no equivalent guarantee is available there.

3.3 Revised SIDS Text: `FlowSolution_t` (§7.7, Note 5)

Current text:

For unstructured zones GridLocation options are limited to Vertex or CellCenter, unless one of PointList or PointRange is present.

Proposed text:

For unstructured zones GridLocation options are limited to Vertex or CellCenter, unless one of PointList or PointRange is present, or unless GridLocation is FaceCenter and the zone has complete NGON face coverage (§7.3), in which case indexing follows the zone’s NGON_n element numbering directly.

3.4 Revised SIDS Text: `DiscreteData_t` (§12.4)

The matching clause in `DiscreteData_t` is amended the same way, for consistency — both notes were introduced together by CPEX 0031 and have always read the same way, modulo word order (“PointRange or PointList” here vs. “PointList or PointRange” in §7.7):

Current text:

For unstructured zones GridLocation options are limited to Vertex or CellCenter, unless one of PointRange or PointList is present.

Proposed text:

For unstructured zones GridLocation options are limited to Vertex or CellCenter, unless one of PointRange or PointList is present, or unless GridLocation is FaceCenter and the zone has complete NGON face coverage (§7.3), in which case indexing follows the zone's NGON_n element numbering directly.

ZoneSubRegion_t and BC_t, the other two structures touched by CPEX 0031, are deliberately *not* amended here. ZoneSubRegion_t always requires an explicit region descriptor (point set, BCRegionName, or GridConnectivityRegionName) regardless of GridLocation, so Note 5 there is not a burden in the same way. BC_t does not carry this clause at all (a BC patch is definitionally a *subset*, so an explicit point set is appropriate there even for a complete-coverage zone). See Section 9.4 for discussion of whether a future CPEX should revisit ZoneSubRegion_t as well.

3.5 Revised DataSize[] Structure Function

Section 7.7's DataSize[] definition gains one new FaceCenter case inside *each* of the two existing Rind branches, in direct parallel with the existing Vertex and CellCenter cases. The explicit PointRange/PointList branch remains first and still takes precedence, so existing identity-point-set files are unaffected; and because the new cases sit *inside* the Rind branches (rather than ahead of them), a FaceCenter solution carrying rind elements is still sized with its rind contribution, exactly as CellCenter is. The redline below follows the existing SIDS pseudocode style (**then** keywords, braces on their own lines) exactly as it appears in the current SIDS source, so the amendment can be applied to the standard's text verbatim; the two new cases are marked:

```

1  if (PointRange/PointList is present) then
2  {
3    DataSize[] = ListLength[] ;
4  }
5  else if (Rind is absent) then
6  {
7    if (GridLocation = Vertex) or (GridLocation is absent)
8    {
9      DataSize[] = VertexSize[] ;
10   }
11   else if (GridLocation = CellCenter) then
12   {
13     DataSize[] = CellSize[] ;
14   }
15   else if (GridLocation = FaceCenter) and           /* new */
16     (zone has complete NGON face coverage) then
17   {
18     DataSize[] = FaceSize[] ;
19   }
20 }
21 else if (Rind is present) then

```

```

22 {
23 if (GridLocation = Vertex) or (GridLocation is absent) then
24 {
25     DataSize[] = VertexSize[] + [a + b,...] ;
26 }
27 else if (GridLocation = CellCenter)
28 {
29     DataSize[] = CellSize[] + [a + b,...] ;
30 }
31 else if (GridLocation = FaceCenter) and /* new */
32     (zone has complete NGON face coverage) then
33 {
34     DataSize[] = FaceSize[] + [a + b,...] ;
35 }
36 }

```

Here `FaceSize[]` is defined as $E_{\text{end}} - E_{\text{start}} + 1$ for the zone's contiguous NGON_n block (Section 3.2), and `RindPlanes = [a,b,...]` as in the existing definition. The name `FaceSize[]`, rather than `NFace[]`, is chosen to avoid confusion with `NFACE_n` cell counts.

Note that §12.4 requires no matching `DataSize[]` amendment: the SIDS defines `DiscreteData_t`'s `DataSize[]` by reference — “It is identical to the function `DataSize[]` defined for `FlowSolution_t`” — so the change above propagates to `DiscreteData_t` automatically. Only the §12.4 *Note* (Section 3.4) needs its own redline.

3.6 MLL Changes

3.6.1 New internal helper

```

1  /* cgns_internals.c */
2  int cgi_zone_ngon_face_range(cgns_zone *zone,
3                               cgsize_t *start, cgsize_t *end);
4  /* Returns CG_OK and the contiguous [start,end] face range if the
5   * zone has complete NGON face coverage per the definition in
6   * this document's Detailed Description section; returns
7   * CG_NODE_NOT_FOUND if the zone has no NGON_n sections at all;
8   * returns CG_ERROR (with cgi_error() detail) if NGON_n sections
9   * exist but coverage is not complete (gaps, or mixed with
10  * classical 2-D face-bearing sections). The cgi_error() message
11  * names the zone and, for the gap case, the interrupting section
12  * (e.g. "zone 'fluid_zone': NGON_n sections [1,20] and [31,50]
13  * are not contiguous (elements 21-30 belong to section 'Mixed')",
14  * so the failure is diagnosable without re-deriving element
15  * ranges by hand. Rejects (CG_ERROR) any ElementRange with
16  * start < 1 or end < start, and any face count that would
17  * overflow cgsize_t arithmetic in the caller's size computation,
18  * before returning -- both ranges are read from the file and
19  * must not be trusted blindly. */

```

This helper is used by `cg_sol_write/cg_field_write` (and their `cg_discrete_*`

counterparts) whenever `GridLocation = FaceCenter` is requested on an unstructured zone with no accompanying point set, to (a) confirm the relaxation applies and (b) compute the expected `DataArray_t` size for validation on write, exactly as `CellSize[]` is already used for `CellCenter`. Coverage is a property of the zone's `Elements_t` section layout, already resident in memory as `zone->section[]`; the reference implementation computes it once per zone (caching the result on `cgns_zone`) rather than re-scanning the section list on every `cg_field_write` call to a multi-field `FaceCenter` solution, and invalidates the cached result on `cg_section_write/cg_section_partial_write`.

A thin public wrapper is also added, so that reader-side code (as in Example 5.3) does not have to reimplement the coverage predicate independently for every downstream tool:

```
1  /* cgnslib.c; public API */
2  int cg_zone_ngon_face_range(int fn, int B, int Z,
3                             cgsize_t *start, cgsize_t *end);
4      /* Thin wrapper over cgi_zone_ngon_face_range() for read-side
5       * consumers. Same return-value contract. */
```

3.6.2 Fix: `cg_sol_write` allow-list

Rather than add `FaceCenter` as one more literal in `cg_sol_write`'s local allow-list, the reference implementation *removes* that allow-list and delegates to `cgi_check_location()` — the same dimension- and zone-type-aware check that `cg_gridlocation_write` and `cg_sol_ptset_write` already trust, and the reason those two entry points already accept `FaceCenter` correctly today while `cg_sol_write` does not. `cg_sol_write`'s own literal allow-list and its separate, hand-written `[IJK]FaceCenter/Structured-only` check are two independent copies of a rule `cgi_check_location` already enforces; keeping both is exactly the kind of duplication that let them drift apart and produced this bug in the first place. Because `cgi_check_location` needs the zone's `CellDimension` and `ZoneType_t`, the check must move to *after* the zone lookup (`cg_sol_write` currently validates `location` before `cgi_get_file/cgi_get_zone` are called):

```
1  /* was: a literal allow-list checked before the zone lookup, plus a
2   * separate structured-only [IJK]FaceCenter check further down. Both
3   * are replaced by one delegated call, placed after the zone lookup: */
4  zone = cgi_get_zone(cg, B, Z);
5  if (zone == 0) return CG_ERROR;
6  if (cgi_check_location(cg->base[B-1].cell_dim, zone->type, location))
7      return CG_ERROR; /* cgi_check_location has already set cgi_error()
8   */
```

This is strictly additive in effect: every `location` value previously accepted by the old allow-list is still accepted (each is one of `cgi_check_location`'s `CG_OK`

cases), plain `FaceCenter` is now accepted for `CellDimension` ≥ 3 regardless of zone type (matching what `cg_gridlocation_write` already permits today), and `[IJK]FaceCenter` on a non-Structured zone is still rejected, with the same error message, by `cgi_check_location` itself. This fix is independent of the rest of this CPEX (it does not by itself relax Note 5) but is required in order for Example 5.2’s single-call write to work; without it, applications must still fall back to the two-step `cg_sol_write` + `cg_gridlocation_write` sequence for the point-set-free form. Being a narrowly-scoped, independently testable bug fix with no dependency on the NGON-coverage logic, it can land as its own patch against the current release line without waiting on the rest of this CPEX; see the note in Section 11.

3.6.3 Fix: `cgnscheck` point-set-aware sizing

`get_data_size()` (Section 2.5.2) is updated to: (1) read a `PointList/PointRange` under `FlowSolution_t/DiscreteData_t`, if present, and use its `ListLength` as the expected size (this branch is required by the *current* SIDS and was simply never implemented); and (2) when no point set is present and `GridLocation` = `FaceCenter` on an unstructured zone, call `cgi_zone_ngon_face_range()` and use the returned count. Only when neither applies does `get_data_size()` retain its current “grid location not valid for unstructured zone” error.

3.6.4 No change to on-disk representation

No new node type, attribute, or data layout is introduced. A conformant file under this CPEX is a `FlowSolution_t` (or `DiscreteData_t`) node with `GridLocation` = `FaceCenter` and *no* `PointList/PointRange` child, on a zone with complete NGON face coverage. Because this is a strict superset of what Note 5 currently forbids, this CPEX changes only the *normative interpretation* of an existing, legal-today node shape — it does not add anything a pre-CPEX HDF5/ADF reader would fail to open at the byte level.

3.7 Scope / Non-Goals

- **Not addressed:** `EdgeCenter` on 2-D unstructured zones. No SIDS element type gives edges a completeness guarantee analogous to `NGON_n`’s for faces.
- **Not addressed:** implicit sizing for classical 2-D sections (`TRI_3/QUAD_4/...`). See Section 3.2.
- **Not addressed:** `ZoneSubRegion_t`, `BC_t`. See Section 9.4.
- **Not required:** no change to `NFACE_n`’s own definition, or to how `CellCenter` is already handled; both are unaffected and serve as the existing precedent this CPEX generalizes.

4 Backward Compatibility

359

- **Existing files:** Fully compatible. Files using the current identity-point-set
workaround remain valid; the explicit `PointRange/PointList` branch of the
revised `DataSize[]` function still takes precedence over the new implicit branch,
so nothing about their interpretation changes.

360
361
362
363
- **Existing readers (pre-CPEX-0051).** A pre-CPEX reader that encoun-
ters a `FaceCenter FlowSolution_t` with no point set on a complete-NGON
zone will, under the *old* SIDS text, consider the file non-conformant (Note 5
as currently written requires the point set), and a naive reader may simply
misinterpret the array size instead of erroring cleanly. This is the one real
compatibility wrinkle, and it deserves emphasis: writers that adopt this CPEX
are producing files that are conformant under the *new* SIDS text but not the
old one, so **a writer must not omit the point set unless it knows its
downstream tool ecosystem supports this CPEX**. CGNS versions its
SIDS compliance via `CGNSLibraryVersion_t` (and, if CPEX 0049 is adopted,
`CGNSMinRequiredVersion_t`). If CPEX 0049 is adopted, this CPEX **must**
be registered in its Feature-Version Registry (CPEX 0049, “Feature-Version
Registry” section) so that AUTO write mode raises a file’s minimum required
version whenever the implicit `FaceCenter` form is used, and so that version-
aware readers reject or warn appropriately rather than silently misreading array
sizes. If CPEX 0049 is not adopted, this CPEX should instead mandate a plain
`CGNSLibraryVersion_t` bump on write, as the minimum viable version-gating
mechanism.

364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
- Why a major release.** This CPEX is targeted at CGNS **v5.0.0**, the next
major release, rather than a point release on the v4.x line. This is not merely
a labeling choice: under CPEX 0049’s revised `cg_open` logic, a library whose
major version is older than the file’s major version is rejected immediately, before
any feature-compatibility check runs (CPEX 0049, “Modified `cg_open` Behavior,”
major-version safety guard). If this CPEX lands in v5.0.0 alongside CPEX 0049,
a v4.x reader opening a v5.0-written file is hard-rejected at that guard and never
reaches the point where it would have to interpret a point-set-free `FaceCenter`
solution at all, eliminating the “silently misreads the data” failure mode entirely
for that population of readers. This is a stronger guarantee than a minor-version
bump could offer under the same mechanism, since CPEX 0049 only hard-rejects
on a major-version mismatch; a same-major, higher-minor file is instead routed
through the feature-aware check, which depends on this CPEX’s Feature-Version
Registry entry being present and correctly detected.

382
383
384
385
386
387
388
389
390
391
392
393
394
395
- **Existing writers:** Unaffected. Codes that do not use the new implicit form
continue to write the explicit point set exactly as before; nothing is deprecated or
removed. *Today*, this ordering independence is unconditional: `cg_field_write`

396
397
398

branches on whether a point set is already attached to the solution at the moment *it* is called (`sol->ptset == NULL`), and writing a `FaceCenter` field before any point set exists already fails on an unstructured zone (no implicit sizing rule exists yet), so no existing writer can depend on a field-before-point-set ordering. This CPEX changes that: once implicit sizing exists, a solution written field-first would size against the NGON face count, and a point set added *afterward* would silently reinterpret an already-on-disk array written to a different, incompatible size. The reference implementation must therefore forbid attaching a `PointList/PointRange` to a `FlowSolution_t/ DiscreteData_t` node that already has one or more `DataArray_t` children written under implicit sizing — returning `CG_ERROR` from the point-set write call in that case — rather than silently permitting the reinterpretation. This is a new ordering constraint introduced by this CPEX, not a preexisting one, and should be covered by its own test case (Section 8).

- **ABI:** No structure layout changes are required beyond the addition of the internal `cgi_zone_ngon_face_range()` helper and its public wrapper `cg_zone_ngon_face_range()` (Section 3.6.1). The public API changes are limited to widening `cg_sol_write`'s accepted `location` values (already a bug fix independent of this CPEX) and adding the new read-side wrapper, and are purely additive in effect (previously-rejected input is now accepted; no previously-accepted input is rejected; the wrapper is an optional convenience with no existing caller). **No new public error codes are introduced.** A write requesting implicit `FaceCenter` sizing on a zone that lacks complete NGON face coverage returns the existing `CG_ERROR` (with a `cgi_error()` message identifying the missing coverage), exactly as an out-of-range `GridLocation` does today; `cg_zone_ngon_face_range()` returns the same `CG_OK/CG_NODE_NOT_FOUND/CG_ERROR` contract as every other existing `cg_*_info`-style read function, so no new return-code semantics are introduced at the public boundary either.
- **Parallel I/O (`cgp_*`):** Unaffected. The relaxation changes only whether a point-set child node must exist; it does not change the collective/independent write model for `DataArray_t` contents.

5 Examples

5.1 Example 1: Before — Identity `PointRange` (current, still valid)


```

1  int S, F;
2  cg_sol_write(fn, B, Z, "CellData", CGNS_ENUMV(CellCenter), &S);
3  cg_field_write(fn, B, Z, S, CGNS_ENUMV(RealDouble), "Pressure",
4                pressure, &F);
5
6  /* FaceCenter WITH an explicit point set already works today in one
7   * call via cg_sol_ptset_write -- the gap this CPEX's reference
8   * implementation fixes (see Existing Implementation Gaps, above) is
9   * specifically the point-set-FREE form below, not this one. The
10  * identity PointRange must still be kept in sync with NGonElements'
11  * ElementRange by hand. */
12  cgsize_t range[2] = {1, 36};
13  cg_sol_ptset_write(fn, B, Z, "FaceData", CGNS_ENUMV(FaceCenter),
14                    CGNS_ENUMV(PointRange), 2, range, &S);
15  cg_field_write(fn, B, Z, S, CGNS_ENUMV(RealDouble), "NormalX",
16                normal_x, &F);

```

5.2 Example 2: After — Implicit FaceCenter (this CPEX)

432

```

1  int S, F;
2
3  /* FaceCenter now accepted directly by cg_sol_write (fix described in
4   * this CPEX's MLL Changes section); no PointRange needed -- the
5   * zone's NGonElements section already defines the face numbering. */
6  cg_sol_write(fn, B, Z, "FaceData", CGNS_ENUMV(FaceCenter), &S);
7  cg_field_write(fn, B, Z, S, CGNS_ENUMV(RealDouble), "NormalX",
8                normal_x, &F); /* size validated against NGON face count
9                               */

```

Resulting tree (compare to the example in Section 2.1):

433

```

1  fluid_zone Zone_t [[27 8 0]]
2  +---ZoneType ZoneType_t "Unstructured"
3  +---NGonElements Elements_t [22 0]
4  |   +---ElementRange IndexRange_t [1 36]
5  +---NFaceElements Elements_t [23 0]
6  |   +---ElementRange IndexRange_t [37 44]
7  +---CellData FlowSolution_t
8  |   +---GridLocation GridLocation_t "CellCenter"
9  |   +---Pressure DataArray_t
10 +---FaceData FlowSolution_t
11 |   +---GridLocation GridLocation_t "FaceCenter"
12 |   +---NormalX DataArray_t /* 36 values; indexed 1..36 against
13                             NGonElements, no PointRange node */

```

The MLL's Fortran bindings (`cg_sol_write_f`, `cg_field_write_f`) forward the same location argument through the same validation as their C counterparts, so both the `cg_sol_write` allow-list fix and this CPEX's implicit-sizing behavior are directly visible to Fortran callers, who make up a substantial share of CGNS writers. The Fortran mirror of Example 5.2:

434

435

436

437

438


```

1 integer :: S, F, ier
2
3 ! FaceCenter now accepted directly by cg_sol_write_f; no PointRange
4 ! needed -- the zone's NGonElements section defines the numbering.
5 call cg_sol_write_f(fn, B, Z, 'FaceData', FaceCenter, S, ier)
6 if (ier .ne. CG_OK) call cg_error_exit_f()
7 call cg_field_write_f(fn, B, Z, S, RealDouble, 'NormalX', &
8     normal_x, F, ier)
9 if (ier .ne. CG_OK) call cg_error_exit_f()

```

5.3 Example 3: Reading — Dispatching on Coverage

439

```

1 char name[33];
2 CGNS_ENUMT(GridLocation_t) loc;
3 CGNS_ENUMT(PointSetType_t) ptype;
4 cgsize_t npnts, fstart, fend;
5
6 if (cg_sol_info(fn, B, Z, S, name, &loc)) return handle_error();
7
8 /* cg_sol_ptset_info addresses solution S directly and reports "no
9 * point set" as CG_OK with ptype == PointSetTypeNull -- unlike the
10 * cg_goto-positioned cg_ptset_info, it needs no prior cg_goto call,
11 * and a real read failure is distinguishable from "point set absent". */
12 if (cg_sol_ptset_info(fn, B, Z, S, &ptype, &npnts)) return handle_error();
13 int has_ptset = (ptype != CGNS_ENUMV(PointSetTypeNull));
14
15 if (loc == CGNS_ENUMV(FaceCenter) && !has_ptset) {
16     /* Node shape alone (FaceCenter, no point set) is necessary but
17     * NOT sufficient: a pre-CPEX-0051 or non-conformant file can
18     * present this same shape without complete NGON coverage. Verify
19     * before trusting it -- this CPEX makes the check cheap, not
20     * optional, which is why cg_zone_ngon_face_range is public. */
21     if (cg_zone_ngon_face_range(fn, B, Z, &fstart, &fend) != CG_OK) {
22         /* Claims the implicit form but coverage doesn't hold:
23         * non-conformant file. Fail safely rather than guessing a
24         * size -- this is the "silently misreads the data" failure
25         * mode Section 4 warns against. */
26         return handle_nonconformant_file(fn, B, Z, S);
27     }
28     process_all_faces(fn, B, Z, S, fend - fstart + 1);
29 } else if (loc == CGNS_ENUMV(FaceCenter) && has_ptset) {
30     /* Explicit form: a genuine subset, or a pre-CPEX-0051 file. */
31     process_face_subset(fn, B, Z, S, ptype, npnts);
32 }

```

5.4 Example 4: Rejected — Non-Contiguous NGON_n Sections

440

```

1  /* Zone has TWO NGON_n sections separated by an unrelated section,
2  * e.g. ElementRange [1,20] (NGON_n), [21,30] (some other type),
3  * [31,50] (NGON_n). Condition 2 of the coverage definition is
4  * violated: the NGON_n ranges do not form a single contiguous block,
5  * so there is no well-defined [E_start, E_end] to index against. The
6  * explicit point set is still required.
7  *
8  * Note: a zone with an NGON_n section PLUS a separate boundary-patch
9  * QUAD_4 section OUTSIDE the NGON_n range (e.g. ElementRange [1,36]
10 * NGON_n, [37,44] QUAD_4 for BC_t) is fine and DOES qualify -- see
11 * the coverage definition's revised Condition 2 and Example 5 below. */
12 cg_sol_write(fn, B, Z, "FaceData", CGNS_ENUMV(FaceCenter), &S);
13 /* cg_field_write size-validation falls back to requiring an
14 * explicit PointList/PointRange, and cgnscheck flags its absence. */

```

5.5 Example 5: Accepted — NGON_n Coexisting with a Boundary-Patch Section

```

1  /* Zone has an NGON_n section [1,36] covering every face, PLUS a
2  * separate QUAD_4 section [37,44] duplicating the boundary faces
3  * for a BC_t patch -- a common pattern for tool compatibility.
4  * This still qualifies for implicit FaceCenter indexing: the
5  * NGON_n range [1,36] is contiguous on its own, and the QUAD_4
6  * section lies entirely outside it. */
7  cg_sol_write(fn, B, Z, "FaceData", CGNS_ENUMV(FaceCenter), &S);
8  cg_field_write(fn, B, Z, S, CGNS_ENUMV(RealDouble), "NormalX",
9  normal_x, &F); /* 36 values, indexed against [1,36] */

```

6 SIDS Impact

- §7.7 FlowSolution_t, Note 5: amended (Section 3.3). 444
- §7.7 FlowSolution_t, DataSize[] function: amended (Section 3.5). 445
- §12.4 DiscreteData_t, equivalent Note: amended (Section 3.4). No 446
DataSize[] amendment is needed in §12.4 itself: the SIDS defines 447
DiscreteData_t's DataSize[] by reference ("It is identical to the func- 448
tion DataSize[] defined for FlowSolution_t"), so the §7.7 change propagates 449
automatically (Section 3.5). 450
- §7.3 Elements_t: unchanged. This CPEX relies entirely on the existing global- 451
numbering and "NGON_n specifies all faces" guarantees already stated there 452
(Section 2.3); no new text is needed in §7.3 itself. 453
- No new SIDS node types, enumerations, or structural fields are introduced 454
anywhere. 455

7 File Mapping Impact

None. This CPEX changes only which combinations of existing nodes are considered conformant; it introduces no new HDF5/ADF on-disk structures, attributes, or data types. The ADFH and ADF file-mapping documents require no amendment.

8 Test Plan

The reference implementation's test suite must cover at least the following fixtures. Items 1–5 exercise the coverage predicate and the new sizing rule directly; items 6–7 pin down behavior this CPEX makes newly reachable and that would otherwise be left implicit; item 8 guards the one existing entry point this CPEX widens.

1. Single `NGON_n` section, implicit form (Example 5.2): round-trip write, read, and `cgnscheck` clean (zero errors) in both C and Fortran (Fortran mirror of Example 5.2).
2. Multiple contiguous `NGON_n` sections partitioning one `[E_start, E_end]` block: accepted, sized correctly.
3. Gap case (Example 5.4): write of a solution requesting implicit sizing is rejected with `CG_ERROR`; a file constructed to have this shape externally is flagged by `cgnscheck` and by `cg_zone_ngon_face_range()` returning `CG_ERROR` on read.
4. Boundary-patch coexistence (Example 5.5): accepted. This is new behavior with no installed base to regress against, so it carries the highest regression value of any fixture here — it was a deliberate relaxation from an earlier draft of this CPEX (Section 3.2) and deserves its own dedicated test rather than incidental coverage from another fixture.
5. Legacy identity-`PointRange` file (Example 5.1): still passes, unchanged size, confirming the explicit-point-set branch of `DataSize[]` retains precedence over the new implicit branch.
6. Point-set-written-after-implicit-fields ordering: attempt to attach a `PointRange/PointList` to a `FlowSolution_t` that already holds fields written under implicit NGON sizing; must return `CG_ERROR` per the rule in Section 4. This behavior is otherwise undefined by the prose alone; the test is what pins it down.
7. Malformed `ElementRange` inputs to `cg_zone_ngon_face_range()`: `start < 1`, `end < start`, and (on a 32-bit `cgsizes_t` build) a range whose face count overflows subsequent size arithmetic — all three must return `CG_ERROR` before any allocation is attempted downstream (Section 3.6.1).

8. Structured zone with plain `FaceCenter` through the fixed `cg_sol_write` (Section 3.6.2): must still be accepted for `CellDimension` ≥ 3 (matching `cg_gridlocation_write`'s existing behavior) and rejected below that, pinning down the delegated-`cgi_check_location` behavior in both directions.

Note on `cgnscheck` regression risk. Once `get_data_size()` is fixed to honor an explicit point set (Section 2.5.2), files that previously produced a false-positive `ERROR:` for *any* reason unrelated to sizing will, for the first time, actually have their declared array size checked against `ListLength[]`. Any existing file whose `FlowSolution_t` array size does not genuinely match its point set's `ListLength` — previously masked by the unconditional unstructured-zone bailout — will newly and correctly fail. This is the widest legacy-system ripple in this proposal: it is a correctness fix, not a regression, but should be called out explicitly in `cgnscheck` release notes so it is not mistaken for one.

9 Design Decisions

9.1 Alternative 1: Broaden Coverage to Any Complete 2-D Section

A more general rule could drop Note 5 whenever *any* face-bearing element type's combined `ElementSize` equals the zone's total face count, regardless of type (`TRI_3/QUAD_4/ NGON_n`). **Rejected.** Nothing in the SIDS guarantees that classical 2-D sections are complete; in the large majority of existing files they represent boundary patches only. This alternative would require either (a) trusting a coincidental size match, which is fragile and has produced silent misinterpretation bugs in similar contexts elsewhere in the ecosystem, or (b) introducing a new explicit “these elements are all the faces” marker — which is just a worse-ergonomics version of what `NGON_n` already means by definition. Scoping strictly to `NGON_n` avoids inventing any new machinery.

9.2 Alternative 2: New Explicit Marker Node (e.g., `FacesComplete`)

Introduce a boolean/flag child of `Zone_t` or `Elements_t` stating explicitly that a section enumerates all faces — reviving, in spirit, the 2005-era “`ZoneElementType`” idea from the original face-based storage proposal that predated `NGON_n/NFACE_n` standardization. **Rejected.** `NGON_n` already carries this meaning by SIDS definition (§7.3); adding a redundant marker node reintroduces exactly the kind of “assert something already implied” overhead this CPEX exists to remove, just relocated from `FlowSolution_t` to `Zone_t`.

9.3 Alternative 3 (chosen): Structural Inference from NGON_n Coverage 523

Infer completeness structurally from the conditions in Section 3.2, using only guarantees the SIDS already makes. No new nodes; no new trust assumptions beyond ones the standard already relies on for `CellCenter`. This is the design adopted by this CPEX. 524 525 526 527 528

9.4 On Scope: Why Not Also Amend ZoneSubRegion_t? 529

`ZoneSubRegion_t` (§7.9) shares CPEX 0031 ancestry with `FlowSolution_t` and `DiscreteData_t`, but it always requires an explicit region descriptor by design — a subregion that happened to cover every face would be indistinguishable from simply using `FlowSolution_t`, so there is no analogous “identity point set” pain point to remove. This is left as an open question for the committee rather than folded into this proposal’s scope. 530 531 532 533 534 535

10 Open Questions 536

1. Should `cgnscheck` *warn* (rather than silently accept) when it encounters the pre-CPEX-0051 identity-`PointRange` pattern on a complete-NGON zone, to nudge writers toward the more compact implicit form? This proposal takes no position; it is a tooling policy choice, not a conformance question. 537 538 539 540
2. Should version gating use CPEX 0049’s `CGNSMinRequiredVersion_t`/feature-mask mechanism, if adopted, or a plain `CGNSLibraryVersion_t` bump? This CPEX assumes whichever mechanism is current SIDS practice at the time of adoption and does not mandate a specific one. 541 542 543 544
3. CPEX 0050 (`DofStorage_t`), Example 4, already writes a `FaceCenter TraceSolution` via a single `cg_sol_write` call with no accompanying point set, for a zone implied to have `NGON_n` coverage. That example is only fully conformant once both this CPEX (for the missing point set) and the `cg_sol_write` allow-list fix (Section 2.5.1) are adopted. If both CPEXes proceed, their reference implementations should be sequenced so CPEX 0051 lands first, or the dependency should be noted explicitly in CPEX 0050. 545 546 547 548 549 550 551

11 Summary of API Additions 552

The two implementation gaps from Section 2.5 (`cg_sol_write`’s allow-list, and `cgnscheck`’s missing point-set read) are pre-existing bugs, independent of the NGON-coverage logic itself, and do not need to wait on this CPEX’s SIDS-level relaxation or on v5.0.0. They can and should land as standalone patches against the current 553 554 555 556

release line as soon as they are reviewed; this CPEX’s reference implementation only *depends on* them, so that Example 5.2’s single-call write works end to end.

Symbol	Change	Release
<code>cg_sol_write</code>	Delegates location check to <code>cgi_check_location</code> , accepting <code>FaceCenter</code> directly (Sec. 3.6.2)	any (backportable)
<code>cg_discrete_write</code>	Same fix, mirrored	any (backportable)
<code>get_data_size()</code> (cgnscheck)	Point-set-aware sizing (Sec. 2.5.2)	any (backportable)
<code>cgi_zone_ngon_face_range()</code>	New internal helper (Sec. 3.6.1)	v5.0.0 (this CPEX)
<code>cg_zone_ngon_face_range()</code>	New public wrapper, read-side coverage check (Sec. 3.6.1)	v5.0.0 (this CPEX)

Aside from the new public `cg_zone_ngon_face_range()` wrapper, no new public enumerations, structs, or function signatures are introduced; `FaceCenter` already exists in `GridLocation_t`.

12 References

1. CGNS SIDS, §7.3 `Elements_t` (element numbering, `NGON_n/NFACE_n` definition). 563
2. CGNS SIDS, §7.7 `FlowSolution_t` (Notes and `DataSize[]` function). 564
3. CGNS SIDS, §12.4 `DiscreteData_t`. 565
4. CGNS SIDS Changelog, Version 3.1 (CPEX 0031: `PointList/PointRange` revamp; `NGON_n/NFACE_n` formalization). 566
567
5. CPEX 0049, CGNS Compatibility Version Control (version-gating mechanism referenced in Section 4). 568
569
6. CPEX 0050, DOF Storage Type (Example 4 dependency noted in Open Question 3, Section 10). 570
571
7. S. Allmaras, “Face-Based Proposal” (2005-era precursor discussion of face-based unstructured storage, predating standardized `NGON_n/NFACE_n`). 572
573